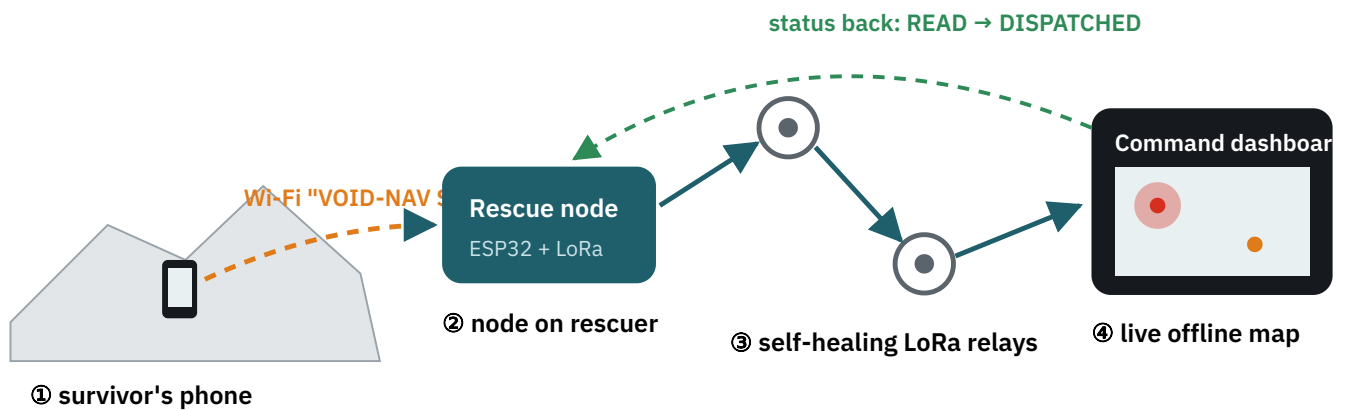


VOID-NAV

An instant private network that rescuers carry in, so trapped survivors can call for help when every tower is down.

One scenario, done perfectly: a survivor is trapped (rubble, flood, collapsed building) and still has a phone. A rescuer arrives with a VOID-NAV node. The node creates a local SOS Wi-Fi, the survivor sends an SOS from their phone browser, and the message travels over a **self-healing LoRa mesh** to a **live offline dashboard** where rescuers read it, dispatch help, and the status flows back to the survivor.



INSTANT

Network exists the moment a rescuer switches a node on.

PRIVATE

Encrypted, authenticated, no identity collected.

SELF-HEALING

Relays find new routes when one dies.

What's inside this plan

1 Reality check: how survivors actually join · 2 Architecture · 3 SOS input (form, button, text, voice, language, compression) · 4 Self-healing mesh · 5 Security & privacy · 6 Quantum: honest integration · 7 Dashboard spec (build-ready) · 8 Hardware: now vs buy · 9 **Tomorrow's demo** · 10 Disasters & other use cases · 11 Business model & future · 12 How to win.

Nothing connects to a phone by itself. Here's what really happens.

Truth #1: a scanner (ESP32 Wi-Fi/BLE) **cannot connect to a phone** or push anything onto it. It can only **listen**. And no node can force a phone to join its Wi-Fi. That's how phones are designed, and it's also a security feature we should be proud of.

Two separate jobs

JOB	HOW IT WORKS	NEEDS SURVIVOR ACTION?
Find phones	ESP32 listens for Wi-Fi "probe requests" phones send while searching for networks	No (only Wi-Fi must be ON)
Talk to survivor	Survivor joins open Wi-Fi "VOID-NAV SOS" ; SOS page opens automatically (captive portal)	Yes: tap the network once

Limits we state honestly

- Phones with Wi-Fi **off**, in airplane mode, or dead are invisible.
- Phones **randomise** their Wi-Fi ID, so we **count** devices, never identify them.
- Bluetooth: most phones don't advertise constantly, so BLE detection is **unreliable** and a bonus only.
- Probe frequency drops when screens are off, so detection takes seconds to minutes.

So how will a survivor know to turn on Wi-Fi and join?

BEFORE THE DISASTER

Public awareness: "In a disaster, **turn Wi-Fi ON and look for VOID-NAV SOS.**" Posters at hospitals, schools, panchayats; part of disaster drills (like "Drop, Cover, Hold").

WHEN THE ALERT GOES OUT

The official warning SMS/cell broadcast adds one line: "Keep Wi-Fi ON. Join VOID-NAV SOS if you see it." (**proposed integration**)

ON SITE

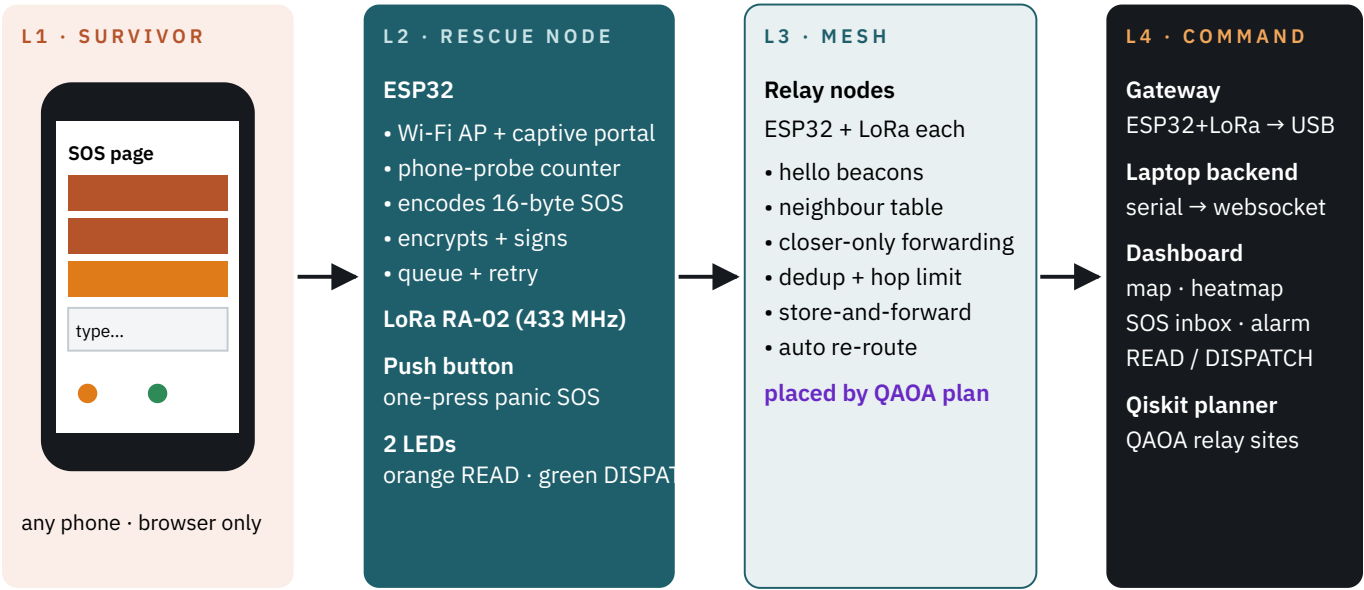
Rescuers **shout / use a loudhailer**: "Turn on Wi-Fi, join VOID-NAV SOS." Survivors often hear voices before they're found.

DEFAULT BEHAVIOUR

Most people keep Wi-Fi on. An open network named **VOID-NAV SOS** appears at the top of their Wi-Fi list; some Android versions also notify "open network available".

Judge answer (memorise): "We don't hack or auto-connect to phones; we can't, and we shouldn't. We **listen** to estimate where phones are, and we **invite** survivors through an open SOS network that opens a help page automatically. Awareness and on-site calls tell them to join."

Four layers, one message journey



One message, end to end

#	STEP	WHERE
1	Survivor joins "VOID-NAV SOS"; the page opens; they tap Trapped , answer 2 questions, add an optional note	phone → node Wi-Fi
2	Node builds a 16-byte core packet (+ optional short text), encrypts it, gives it a unique ID	ESP32
3	LoRa transmits; relays forward toward the gateway using closer-only rules	mesh
4	Gateway verifies, decrypts, sends JSON over USB; the dashboard shows it UNREAD and an alarm sounds if critical	laptop
5	Rescuer opens it → READ (locked) → LoRa status → node orange LED + phone page "Read by rescue"	reverse path
6	Rescuer presses Dispatch → DISPATCHED → node green LED + "Help is on the way"	reverse path

Note: we follow your state mapping: orange = READ by rescuer, green = DISPATCHED. The phone page also shows "Sent ✓" and "Delivered ✓" as text, so survivors aren't left wondering before a rescuer reads it.

Form first, words optional, meaning always

The SOS page (no OLED needed; the phone is the screen)

1. **Big buttons:** Trapped · Injured · Need rescue · Food/water · Safe
2. **Yes / No / Not sure:** Anyone injured? Bleeding? Child or elderly with you?
3. **Count:** 1 · 2–5 · 6+
4. **Where are you?** Ground floor / upper floor / basement / outside (structured)
5. **Optional note** (≤ 60 characters): typed in any language
6. **SEND** → status shows Sent → Delivered → Read → Dispatched

Push button on the node: one press = instant "Need rescue, unknown details" SOS for someone who can't use a phone or reach the screen.

Should survivors speak (voice)?

Not over the network. Browsers block microphone access on plain local (non-HTTPS) pages, and audio is far too big for LoRa. **Workaround that works today:** the phone keyboard's mic button (dictation) turns speech into text on many phones. They speak; it types the note.

Is multilingual needed?

Yes, and it's almost free. Buttons are icons + Telugu / Hindi / English labels. Answers are sent as **codes, not words**, so a Telugu tap shows up in English (or any language) on the dashboard. Free text travels as typed (UTF-8).

Do we need "AI semantic compression"?

For the form: no. The form already *is* the meaning: "trapped, 3 people, injured, basement" becomes a few bytes of codes. That's true semantic encoding, without any AI. **For messy free text:** the ESP32 can't run an AI model, so we don't pretend it does. We do two honest things: (1) **send the person's words as they are** (short, never thrown away), and (2) **keyword tagging** on the node (e.g. "blood", "khoo", "trapped", "water") that raises priority. Optional at the dashboard: an offline language model on the laptop summarises long notes for rescuers.

How to say it to judges: "We transmit meaning as structured codes, plus the survivor's own short words. We never replace what they said with a guess."

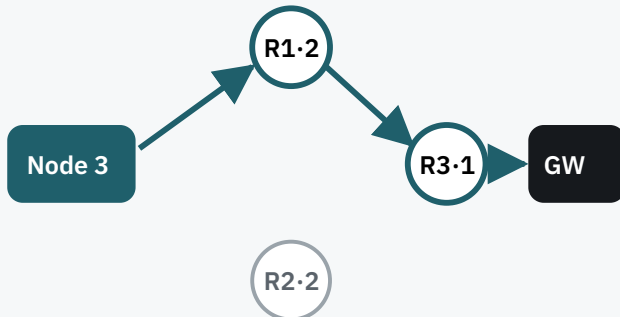
Packet: 16-byte core + optional text

FIELD	BYTES	NOTES
Message ID (random)	4	dedup + matching READ/DISPATCH replies
Node ID (anonymous)	2	which rescue node, not which person
Type · priority · hops left	1	SOS / STATUS / HELLO; P0–P3; max 4 hops
Category · injured · count · position	3	all form answers as codes
Flags + keyword tags	1	child, elderly, bleeding...
Time offset · sequence	2	ordering, expiry
Auth tag (truncated)	3	rejects fake/modified packets
<i>Optional text</i>	0–60	survivor's own words, encrypted

How the relay network repairs itself

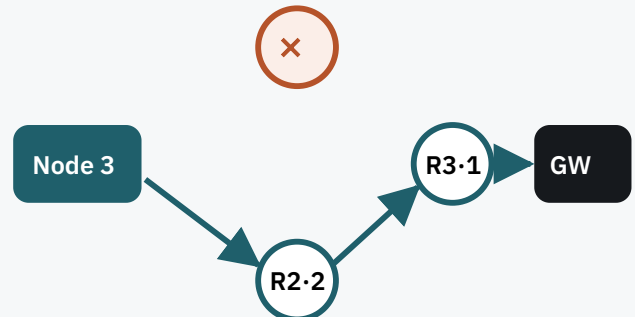
LoRa alone is just a radio. "Self-healing" comes from rules we write in firmware.

Before: route via R1



number = hops to gateway

R1 dies → re-route via R2 automatically



R1 missed 3 beacons → removed from table

1 • HELLO BEACONS

Every node broadcasts a tiny "I'm alive, I'm N hops from the gateway" every ~30 s (randomised so beacons don't collide).

2 • NEIGHBOUR TABLE

Each node remembers who it heard, their hop count and signal quality. Miss 3 beacons → neighbour removed.

3 • RECOMPUTE DISTANCE

My hops = best live neighbour + 1. If the old path dies, a new best path appears within ~1–2 beacon periods.

4 • CLOSER-ONLY FORWARDING

A node forwards an SOS only if it's closer to the gateway than the sender. Several can qualify, so there are **multiple paths** by design.

5 • HOP-BY-HOP RECEIPT + RETRY

No receipt from the next hop within ~2 s → retry (max 3) → try another neighbour.

6 • STORE-AND-FORWARD

No route at all? Keep the SOS (urgent first) until a neighbour reappears; drop it only after it expires (30 min).

ANTI-JAM RULES (SAME AS BEFORE)

Duplicate IDs dropped • max 4 hops • urgent-first queues • listen-before-talk + random wait • capped retries • beacons slowed when the channel is busy.

BE CAREFUL WITH WORDS

Say "**designed for high reliability**" (receipts, retries, multiple paths, buffering), not "100 % reliable". Re-route time and delivery rate are **to be measured**.

A private network that protects survivors, and can't be abused

Protect the survivor's identity

- **We never collect phone numbers.** A Wi-Fi page simply can't read them, and we don't ask.
- **Name is optional.** Messages are tied to an anonymous node ID, not a person.
- **Phone counting doesn't store MAC addresses.** They're hashed with a salt that changes every hour and deleted after the incident. Phones already randomise them anyway.
- **Local IP addresses are not shown.** They're temporary (192.168.4.x) and identify nothing.
- **Data minimisation:** keep only what rescuers need; wipe after the operation.

Protect the network itself

- **Encryption on every LoRa packet** (AES, per-deployment key loaded by the agency). Eavesdroppers see noise.
- **Authentication tag** on every packet: forged or replayed messages are rejected, so no fake SOS floods.
- **Closed network:** only nodes holding the deployment key can relay. Random LoRa devices can't join.
- **Dashboard login** for rescuers; role-based (operator / viewer).
- **Rate limits** per node so one phone can't spam the channel.

On "terrorists use LoRa mesh too": yes, any radio can be misused, and that's exactly why VOID-NAV is **closed, authenticated and deployed by authorities**, not an open public mesh. Encryption protects *survivors* from being tracked; the authority holds the keys and the logs. That's the same principle police and army radios already follow.

SURVIVOR-SIDE SAFETY

The SOS page is served by the node, works offline, asks nothing personal, and the open Wi-Fi only reaches the node, not the internet. A scam or fake portal can't hijack the LoRa network without the key.

RADIO RULES

RA-02 runs at 433 MHz. In India, short-range devices around 433–434 MHz are licence-exempt only at **low power**, so **verify the current WPC limit** and run the demo at reduced power (the RA-02 can be set well below 18 dBm).

Where quantum truly fits, and where it doesn't

Straight answer: the radio, the mesh and the SOS flow are classical, and must be. Microcontrollers in the field can't run quantum programs, and a disaster zone has no internet to reach a quantum computer. Quantum fits in **planning and security**, done on the command laptop or before deployment.

INTEGRATION	WHAT IT DOES	REAL?	USE IT?
1 • QAOA relay placement	Chooses where to put K relays to connect the most survivors (from the heatmap). A real "choose K of N" optimisation → QUBO → QAOA on Qiskit.	Real, runs on laptop (Aer)	YES: headline
2 • Quantum-resistant encryption	AES-256 keeps ~128-bit security even against Grover's algorithm. Messages captured today stay safe when quantum computers mature.	Real (classical crypto, quantum-aware choice)	YES: one line
3 • Quantum random keys	Generate deployment keys from measured qubits (Hadamard + measure on IBM Quantum) before a disaster.	Real but marginal	optional mention
Quantum on the node / "quantum mesh"	—	Not real	Never claim

Will QAOA really work?

Yes, at small scale. We already ran it: 6 candidate sites, place 2 relays, $p = 2 \rightarrow$ QAOA picked **sites 2 + 3 (13 of 21 people)**, the same answer as checking every option. Simulated circuits handle up to ~20 qubits on a laptop.

The catch: at this size a classical solver is just as fast, and QAOA gives approximate answers. Our dashboard runs **both** and shows they agree. Pitch it as "the method that scales as quantum hardware grows", not a speed-up today.

Exact setup needed (install tonight)

```

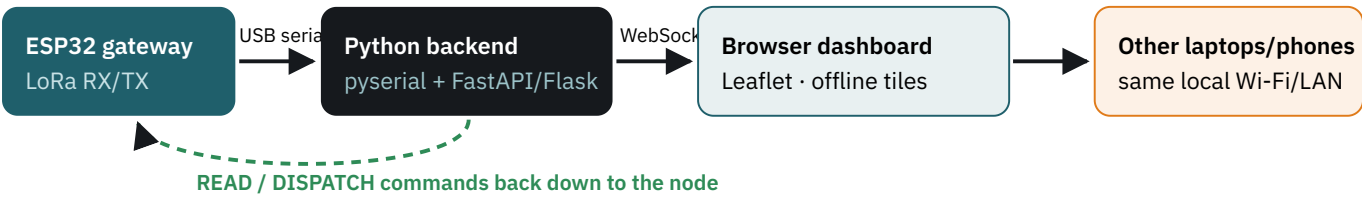
pip install qiskit qiskit-aer qiskit-optimization scipy numpy

Inputs  : candidate relay sites (lat/lon) + survivor clusters (heatmap cells, weight = people)
Model   :  $x_i = 1$  if relay at site  $i$ 
          maximise  $\sum \text{people covered (within radio range of a chosen relay)}$ 
          penalty  $P \cdot (\sum x_i - K)^2 \rightarrow$  QUBO (qiskit-optimization QuadraticProgram)
Solve   : QAOA ( $p=2$ , COBYLA) on AerSimulator vs exact NumPyMinimumEigensolver
Output  : chosen sites drawn on the dashboard map as "planned relays" + coverage circles
Time    : 6–12 sites solve in seconds; button "Plan relays (QAOA)" in the dashboard

```

Optional: run the same circuit once on IBM Quantum (free plan, needs internet) **before** the demo and show the result screenshot as "on real quantum hardware".

Real hardware data, live, offline



Tab 1 • Map

- Offline map of the area (pre-cached tiles)
- **Heatmap** coloured green → yellow → red by SOS count × people
- Node + relay positions, link lines coloured by signal (RSSI/SNR)
- Relay health: online / missed beacons / offline
- "Plan relays (QAOA)" button → planned sites overlay

Tab 2 • SOS inbox

- Cards sorted: **critical first**, then newest
- Shows: exact message text + category + answers + node + hops + signal + time
- **States:** UNREAD → READ (locked) → DISPATCHED
- Dispatch form: team / ambulance / hospital assigned
- **Audible alarm + flashing red** for critical, until opened

Message format: gateway → laptop (one JSON per line)

```
{ "type": "sos", "id": "A7F3C201", "node": "N01", "prio": 0, "cat": "TRAPPED", "injured": "YES", "people": "2-5",  
  "pos": "BASEMENT", "flags": ["BLEEDING"], "text": "leg stuck, mother with me", "hops": 1,  
  "rssi": -48, "snr": 9.5, "ts": 1760000000, "loc": { "lat": 17.385, "lon": 78.486, "src": "CONFIGURED_DEMO" } }  
{ "type": "hello", "node": "R2", "hops_to_gw": 1, "rssi": -71, "batt": 82 }  
laptop → gateway: { "type": "status", "id": "A7F3C201", "state": "READ" } then "DISPATCHED"
```

Real vs mock: label it on screen

SHOWN ON DASHBOARD	SOURCE IN TOMORROW'S DEMO
Message text, category, answers, ID, time, hops, RSSI/SNR	REAL from LoRa hardware
READ / DISPATCHED status reaching the node LEDs	REAL over LoRa
Node location	CONFIGURED (typed in, no GPS). Badge: "demo location"
Background heatmap of a city + other SOS pins	SIMULATED scenario data, toggle "Demo data"
Extra relays on the map	SIMULATED (only 2 radios exist)

Connecting hospitals, agencies, ambulances: the command laptop serves the dashboard on its own local Wi-Fi, so ambulance crews and a field hospital nearby open it on any phone (viewer role). When any backhaul exists (satellite terminal, surviving fibre or cellular), export or forward to district control rooms and 108 dispatch. That integration is **proposed**.

Designed around 1 ESP32, ready for more

Key fact: a relay needs its **own** microcontroller and radio. With one ESP32, both RA-02s sit on the same board, so we can show a **real over-the-air link in both directions**, but not an independent relay. We show relaying/self-healing in a **simulator that uses the same packet format**, and say so.

Wiring: two RA-02 on one ESP32 (shared SPI)

SIGNAL	RA-02 A (NODE TX)	RA-02 B (GATEWAY RX)
3.3 V / GND	3.3 V only (never 5 V)	
SCK / MISO / MOSI	GPIO 18 / 19 / 23 (shared)	
NSS (CS)	GPIO 5	GPIO 4
RESET	GPIO 14	GPIO 27
DIO0	GPIO 26	GPIO 25
Push button	GPIO 33 → GND (internal pull-up)	
LED orange / green	GPIO 32 / GPIO 13 via 220 Ω resistors	

Antennas on before any transmit. Same frequency (e.g. 433 MHz) on both. Keep power low for indoor demo. Add 220 Ω resistors for the LEDs if you don't have them.

Buy list (priority order)

#	ITEM	WHY	PRIORITY
1	ESP32 ×2	separate node and gateway; with a 3rd radio, a true relay	MUST
2	RA-02 + antenna ×1 (same band)	third radio = real 3-node relay chain + unplug-to-heal demo	MUST for "self-healing"
3	220 Ω resistors, buzzer	LED safety; audible alert at node	high
4	Extra power bank / 18650 + holder	run nodes untethered across the room	high
5	NEO-6M GPS	real node coordinates on map	nice
6	Solar panel + TP4056/charge module	"self-powered relay" story; only if a charge board comes with it	nice

"RF SENSOR / BLE SCANNER"

You probably **don't need it**: the ESP32 already scans Wi-Fi probes and BLE. A separate one only helps if it does something extra (e.g. more range).

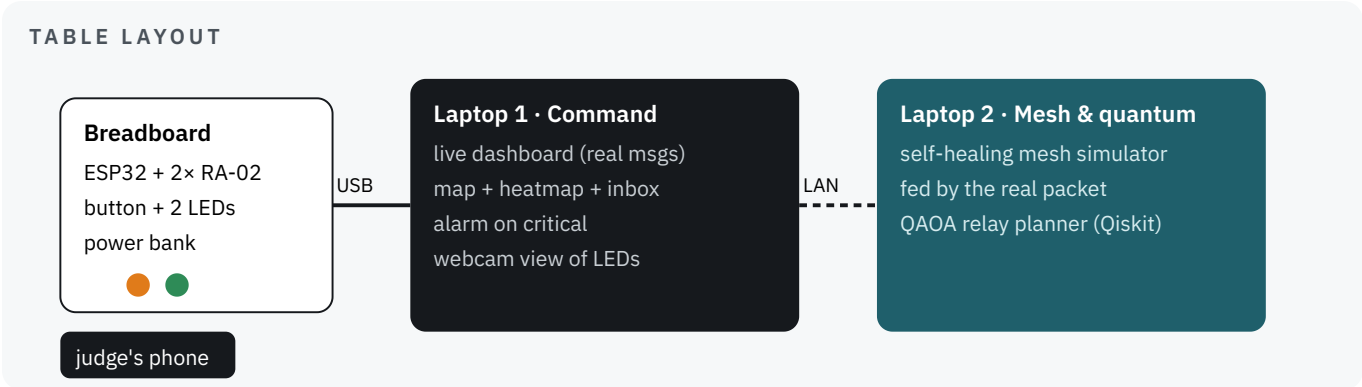
What we can do right now

- ✓ Phone → Wi-Fi SOS page (captive portal)
- ✓ Real LoRa transmit A → receive B, over the air
- ✓ Gateway → USB → live dashboard (real message)
- ✓ READ / DISPATCH back over LoRa → LEDs
- ✓ Push-button panic SOS
- ✓ Phone counting by Wi-Fi probe sniffing (ESP32 itself, no extra sensor) as a separate mode
- ✗ Independent relay (needs another board)
- ✗ Real GPS (no module): use configured location

BALLOON DEMO?

Skip it. Normal (air-filled) balloons don't float. Helium party balloons lift ~10–14 g each; a node (~150–200 g) needs 15–20 of them. Instead, **mount the relay high** on a pole, tripod or shelf and say "in the field: pole, rooftop or tethered balloon".

The 4-minute demo that makes judges remember us



Script (with the hardware we have today)

TIME	ACTION	WHAT JUDGES SEE
0:00	Hook: "Your phone says No Service. You're under a wall."	—
0:30	Hand a judge your phone: "Join VOID-NAV SOS "	SOS page pops up by itself, no internet
1:00	Judge taps Trapped → Yes injured → 2–5 → types "leg stuck"	"Sent ✓" on phone
1:15	Point at Laptop 1	Card appears instantly, alarm sounds , red pin, real text + RSSI
1:40	Rescuer clicks the card → READ	Orange LED lights (webcam on screen) + phone shows "Read"
2:00	Click Dispatch → Ambulance 2	Green LED + phone: "Help is on the way"
2:20	Press the push button	Instant panic SOS appears, critical
2:40	Laptop 2: same packet hops in the simulator; kill a relay	Route re-forms through another relay
3:10	Click "Plan relays (QAOA)"	Qiskit picks sites; matches the exact answer
3:40	Close: privacy, encryption, "instant private network"	—

TIER 1 · TODAY'S KIT

1 ESP32: real 2-way LoRa link, phone SOS, live dashboard, LEDs, button. Relay = simulator.

TIER 2 · +1 ESP32

Node and gateway on separate boards, across the room. Much more convincing.

TIER 3 · +2 ESP32 +1 RA-02

Real relay in the middle. **Unplug it** → message waits → plug in → delivered.

Backups: record a video of the full flow tonight · keep the simulator able to run with "replay last real packet" · print a one-page leaflet · have a second phone already connected.

One instant private network, many emergencies

DISASTER	HOW VOID-NAV IS USED	SPECIAL CARE
Earthquake / building collapse	Rescuers walk rubble with a node; phone count + SOS page locate trapped people; relays on poles at site edges	aftershocks: keep relays away from damaged walls
Flood	Nodes on rescue boats; relays on rooftops/water tanks; rooftop survivors join SOS Wi-Fi	waterproof boxes; boats move, so the mesh keeps re-routing
Cyclone	Relays pre-placed at cyclone shelters before landfall; network ready when towers drop	low, wind-safe mounts; batteries pre-charged
Landslide	Relay chain along a valley road to reach cut-off villages	terrain blocks signal: QAOA planner picks hilltop sites
Fire / industrial accident	Trapped workers report floor and headcount	smoke doesn't block radio; nodes outside the hazard zone
Tunnel / mine collapse	Relay chain carried in behind the rescue team	hop limit raised; spacing tested on site

Beyond disasters: same kit, different mode

DEFENCE

Instant private network for a unit where there's no infrastructure; encrypted, closed, no emissions to the internet.

CROWD MANAGEMENT

Kumbh, stadiums, pilgrimages: lost-person reports and medical calls when cell networks jam from overload.

REMOTE & RURAL

Villages and forests without coverage; forest guards; fishermen near the coast.

Second demo you can show (same hardware): switch the portal to "**Crowd mode**": the page title becomes "Lost / Medical help at the venue", and the dashboard shows the same inbox and heatmap. Thirty seconds, and it proves the "instant private network" platform idea. Keep disaster as the main story.

From hackathon to a standard rescue kit

Business model

STREAM	CUSTOMER
Rescue kits (nodes + relays + gateway)	NDRF, SDRF, fire services, district disaster authorities
Software licence (dashboard, planner, updates)	state disaster authorities, annual
Pre-installed SOS points	hospitals, panchayats, railways, schools (govt + CSR funded)
Event rental (crowd mode)	large event and pilgrimage organisers
Defence variant	via iDEX / defence innovation programmes

Pricing to be validated with real BOM quotes and pilot feedback; we don't quote numbers we haven't measured.

Future development

1. GPS on every node; solar-powered relays
2. Waterproof, drop-tested enclosures
3. Drone / tethered-balloon relays for large areas
4. Satellite backhaul at the command post
5. Offline AI summarisation of long notes at the dashboard
6. Integration with official alerts (CAP/SACHET) and 108
7. Larger QAOA planning on IBM Quantum hardware
8. Pilot drill with an SDMA

How to land top 3 out of 290

1. **Make the judge the survivor:** their phone, their SOS, their green light.
2. **Audible alarm** when the SOS lands. Rooms remember sound.
3. **Webcam on the LEDs** so everyone sees orange → green on the big screen.
4. **Kill a relay** in the simulator: self-healing, visible.
5. **Honesty slide:** real vs simulated, labelled. Judges trust it.

6. **One sentence:** "An instant private network rescuers carry in, so a phone with no signal can still call for help."
7. **Privacy answer ready:** no numbers, no IDs, encrypted, closed.
8. **Quantum answer ready:** planning, not radio; matches the exact answer.
9. **Everyone speaks;** transitions rehearsed.
10. **Backup video** queued in another tab.

Tonight's priority order: (1) LoRa A → B link working, (2) captive portal + form → packet, (3) serial JSON → dashboard inbox with alarm, (4) READ/DISPATCH back → LEDs, (5) map + labelled heatmap, (6) mesh simulator + QAOA button, (7) record backup video.

"When the towers fall, VOID-NAV turns every phone into a beacon and every rescuer into a network."